ESE 507 Advanced Digital System Design and Generation
Fall 2026

Stony Brook University

Part 3: 2D Convolution with Bias and Stride

AI Policy Reminder

Reminder: please see the project AI Policy in Appendix A of the Project Overview document, which explains how AI may and may not be used in this project, and how you should log and disclose AI usage.

The goal of Part 3 is to design a hardware system that performs two-dimensional convolution with a bias and a stride by integrating the components you designed in Parts 1 and 2 and adding additional control logic. (You will also use a FIFO module that we will provide you.) Recall that Figures 4 and 5 in the Project Overview document illustrate the top-level ports and a high-level block diagram for this top-level system. Your system's top-level module should be named `Conv`, and it has parameters `WIDTH`, `R`, `C`, and `K`, as defined previously. (For the legal range of these parameters, please see Project Overview Section 4.2.)

Figure 1 illustrates a partial view of how this module is constructed. Here, you can see that the top-level AXI-Stream input interface directly connects to the Input Memory module, and the AXI-Stream output interface directly connects to an Output FIFO we will provide you. The diagram also shows how the sub-module parameters are set. Most are obvious: `WIDTH`, `R`, `C`, and `K` in the sub-modules should match the corresponding parameters in the top-level module. The MAC's `ACCW` parameter and FIFO's `DEPTH` parameter will be computed as `localparam`s based on your other parameters. (Details are given in Section 3.1 and Section 3.2, respectively.) Also note that the MAC unit is the pipelined `mac_pipe` system you designed in Part 1.2.

Your job is to design the control logic and interconnections that will allow these three modules to work together to perform 2D convolution with bias and stride. Your new logic will need to interact with the signals that are unconnected in this diagram.

Your system will take as inputs the W and X matrices in row-major order, the bias B, the value of Q, and the stride S (all of which you have already handled in Part 2). Then, it will use the MAC module to

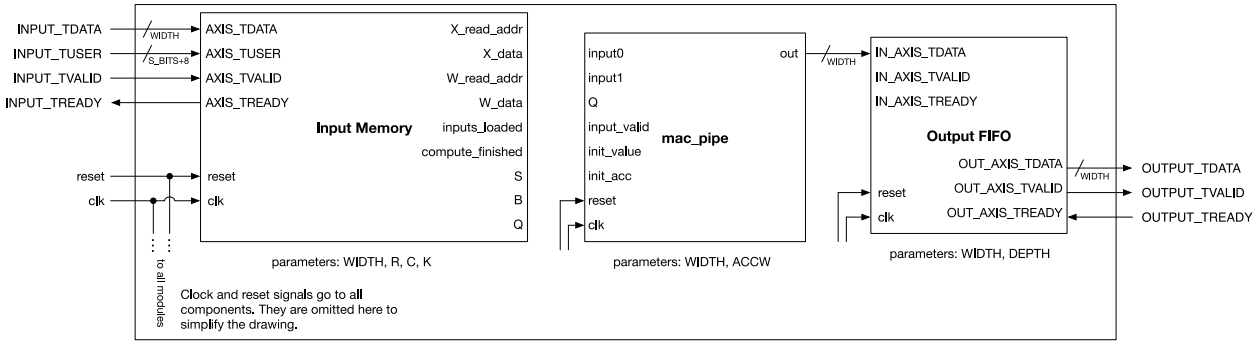


Figure 1: A partial view of how your top-level `Conv` module should be constructed. You will need to design logic to interact with the signals that are unconnected in this diagram. (You may need to zoom in on the PDF to read this diagram clearly.)

compute the values of Y (also in row-major order) and store them into the Output FIFO. The Output FIFO will then provide them to the output AXI-Stream.

Recall from Project Overview Section 3.3 that the `Conv` module must perform the computation seen in the following pseudocode, where $R_{\text{out}} = \left\lfloor \frac{R-K}{S} \right\rfloor + 1$ and $C_{\text{out}} = \left\lfloor \frac{C-K}{S} \right\rfloor + 1$:

```
for r = 0 to Rout-1:
    for c = 0 to Cout-1:
        Y[r][c] = B

        for i = 0 to K-1:
            for j = 0 to K-1:
                Y[r][c] += X[r*S+i][c*S+j] * W[i][j]
```

Since you will be computing and outputting the values of the Y matrix one at a time in row-major order, we can slightly augment this pseudocode to more closely relate to your system:

```
for r = 0 to Rout-1:
    for c = 0 to Cout-1:
        accumulator_reg = B

        for i = 0 to K-1:
            for j = 0 to K-1:
                accumulator_reg += X[r*S+i][c*S+j] * W[i][j]
```

After finalizing the value in the accumulator, your `mac_pipe` logic will scale and saturate it (producing the MAC module output), and store it in the Output FIFO.

Your goal in this task is to connect your understanding of this pseudocode to the hardware components in your system. The X and W matrices can be read from the memories in the Input Memory module, and the scalar values B, Q, and S directly come from the Input Memory module's corresponding outputs. The computation is performed by the MAC module (including the scaling and saturation of the accumulated value), and the final computed values will be written into the Output FIFO. Your control logic will be responsible for making this happen.

1. Module Operation

Here we will give a brief overview of this module's phases of operation and the sequence of steps it will take. You must construct control logic that will make the system undergo the following steps. Your control logic will likely need to include an FSM and counters. The following phases are not necessarily the only FSM states your system will use, but you should use them as a guide for how your control logic for this module should operate.

1. First, the Input Memory module (which you built in Part 2) will interact with the input AXI-Stream interface to load the data into the `X` and `W` memories and store the values of S, B, and Q into appropriate registers. When the input values are loaded and available, the Input Memory module will assert its `inputs_loaded` output signal.

2. Then, your new Part 3 control logic will be responsible for reading the input data from the input memories using the `X_read_addr` and `W_read_addr` signals and feeding the values of X, W, and B into the MAC module. Refer to the pseudocode above to think about how the matrix addressing should work.

   Also recall that the matrices will be stored in the memories in row-major order, so you will need to map accesses like `X[r*S+i][c*S+j]` and `W[i][j]` to addresses. This simply means that `W[0][0]` is address 0, `W[0][1]` is address 1, `W[0][K-1]` is address $K-1$, `W[1][0]` is address K, and so on. X will work similarly, but keep in mind that X has R rows and C columns. (If this isn't clear, work out an example on paper of the addressing with small values for R, C, K, and S.)

   Hint: make sure you design your address logic in a way that avoids expensive and unnecessary logic, especially unneeded dividers and multipliers. (It is possible to design this logic with counters to avoid expensive operations like division, mod, multiplication by S, and others. Think carefully here about how to make your design efficient.)

3. As your system feeds the matrix values into the MAC unit, the MAC will perform the multiply-accumulate operation. Your logic will need to control the MAC module's control inputs (`init_acc`, `init_value`, `Q`, and `input_valid`).

4. The MAC unit's output will connect to the Output FIFO's input signal. As the FIFO receives output data, its internal logic will send output data onto the output AXI-Stream interface.

   The FIFO uses AXI-Stream for its inputs and outputs. So, your control logic will need to interact with the FIFO's `IN_AXIS_TVALID` and `IN_AXIS_TREADY` signals appropriately. If the FIFO's `IN_AXIS_TREADY == 0`, then the FIFO is full, and you cannot write data to it. In this case, your logic would need to stall until the recently computed MAC output can be stored in the FIFO.

5. After your system is done computing the full convolution, the control logic should set `compute_finished` (the input signal to the memory module) to 1 for one clock cycle. This will cause the `input_mems` module to start reading in new inputs again, and the whole process can repeat.

2. The Output FIFO

This design uses a FIFO to hold output values computed by the MAC module before they are sent out of your system. You may be wondering why our system needs to use this FIFO at all. Certainly, it would be possible for the output values produced by the MAC unit to directly go to the AXI-Stream output interface. The downside to such a design would be that the timing of the computation would then depend heavily on the timing of the external `TREADY` control signal. In other words, if the testbench (or whatever real-world component receives data in a real system) sets the output `TREADY` to 0 at the wrong time, you would have to make your entire system stall, wasting time.

Instead, by using a simple FIFO, we can prevent this from happening. As long as there is available space in the FIFO, your system can compute values and store them; the FIFO's internal logic can then place available values on the AXI-Stream output whenever the testbench (or other downstream component) is ready for them.

You are provided with the Output FIFO module. You can find it at:

```
/home/home4/pmilder/ese507/proj/part3/fifo_out.sv
```

Copy this file into your `part3/` work directory. ***For Part 3, you must use this FIFO module without modification.*** (Copying the code into another `.sv` file is also fine if that is convenient for you.)

The FIFO has the following interface:

```
module fifo_out #(
  parameter WIDTH = 16,
  parameter DEPTH = 34,
  localparam LOGDEPTH = $clog2(DEPTH)
)(
  input clk,
  input reset,
  input [WIDTH-1:0] IN_AXIS_TDATA,
  input IN_AXIS_TVALID,
  output logic IN_AXIS_TREADY,
  output logic [WIDTH-1:0] OUT_AXIS_TDATA,
  output logic OUT_AXIS_TVALID,
  input OUT_AXIS_TREADY
);
```

A few things to note:

- The `IN_AXIS_*` and `OUT_AXIS_*` signals are ordinary AXI-Stream interfaces. Your control logic will interact with the `IN_AXIS_*` signals to place computed output values in the FIFO. The FIFO's `OUT_AXIS_*` signals will connect directly to `Conv`'s AXI-Stream output ports to form the primary system outputs.
- The AXI-Stream interface operates in the same way we have previously seen. An input value (on `IN_AXIS_TDATA`) is stored in the FIFO on a positive clock edge if `IN_AXIS_TVALID` and `IN_AXIS_TREADY` are both 1. Similarly, the FIFO puts its output value on `OUT_AXIS_TDATA`; if `OUT_AXIS_TVALID` and `OUT_AXIS_TREADY` are both 1 on a positive clock edge, then the value has been popped from the FIFO.
- The `IN_AXIS_TREADY` signal is driven by the FIFO. It is always equal to 1 unless the FIFO is full. (If the FIFO is full, it cannot hold any more data and thus it is not ready.)
- The `OUT_AXIS_TVALID` signal is driven by the FIFO. It is set to 1 when there is valid data that can be read on `OUT_AXIS_TDATA`. Otherwise it is 0 (so if there is no data stored in the FIFO, the output is not valid).

3. Computed Local Parameters

As described above, most of the submodule parameters will be set based on top-level parameters. (Specifically, `WIDTH`, `R`, `C`, and `K` of the submodules will match those parameters in the top-level `Conv` module.) However, there are two other parameters whose values are computed as `localparam`s based on the other parameters of your system.

3.1. `ACCW`: The Accumulator Width

`ACCW` is the width of the MAC unit's accumulator. In Part 1, we simply used `ACCW` as a parameter you could set. However, now (in Part 3) we know the W matrix dimensions ($K \times K$) that will be used for the convolution. This means we can calculate the smallest value of `ACCW` that is large enough to ensure that the accumulator *cannot possibly* overflow given the computations it will need to perform.

Each convolution will perform K^2 accumulations. This means the MAC's accumulator will compute the sum of K^2 products plus one `WIDTH`-bit bias value. The number of bits required to guarantee you can store that result in your accumulator can be calculated by the following `localparam`. Use this expression in your module:

```
localparam ACCW = $clog2(K*K*(128'd1 << (2*WIDTH-2)) +
(1<<(WIDTH-1)))+1;
```

In Part 1, we constrained `ACCW` to be ≤ 80 bits, and that constraint still applies here. In principle this makes some combinations of `K` and `WIDTH` illegal, but in practice the limit is very hard to reach: even at the largest legal `WIDTH` of 31, `K` can be as large as 724 before `ACCW` exceeds 80, and at `WIDTH=24` it can be over 90,000. (We certainly aren't going to be convolving with a 724×724 W matrix.) The testbench checks the limit anyway and will stop with an error message if you somehow choose an illegal combination.

3.2. `DEPTH`: The Output FIFO's Depth

The Output FIFO's `DEPTH` parameter should be set to $C - K + 1$. This means the FIFO is large enough to fully hold one row of the output matrix.

There is a trade-off here: when this FIFO is large, it reduces the chance that your convolution module will need to stall if the `OUTPUT_TREADY` signal is low. However, larger FIFOs have higher cost. As a reasonable compromise, we have chosen to set it to hold at least one row of the output matrix Y. (Recall that each row of Y will have $C_{\text{out}} = \left\lfloor \frac{C-K}{S} \right\rfloor + 1$ entries, which is largest when S=1, giving $C - K + 1$ entries per row.)

3.3. Other Local Parameters

This module will use `MAXS` and `S_BITS` as defined and used in Part 2. See Section 4.1 of the Part 2 document for a reminder of their meaning. The definitions are also repeated in the code snippet below in Section 4.

4. Code

Store all of your files for Part 3 in a subdirectory called `part3/`. You will need to copy your Parts 1–2 designs here as well, since Part 3 uses them, along with the provided `fifo_out.sv`.

Implement your top-level `Conv` module in a file called `Conv.sv`. Feel free to use multiple files and organize your control logic and other modules in any way that makes sense to you, but make sure everything is commented and named clearly.

Use the following top-level module name, port names, and port declarations:

```
module Conv #(
  parameter WIDTH = 24,
  parameter R = 16,
  parameter C = 17,
  parameter K = 9,

  localparam ACCW = $clog2(K*K*(128'd1 << (2*WIDTH-2)) +
(1<<(WIDTH-1)))+1,
  localparam MAXS = ((R > C) ? R : C) - K + 1,
  localparam S_BITS = $clog2(MAXS+1)
)(
  input clk,
  input reset,
  input [WIDTH-1:0] INPUT_TDATA,
  input INPUT_TVALID,
  input [S_BITS+7:0] INPUT_TUSER,
  output INPUT_TREADY,
  output [WIDTH-1:0] OUTPUT_TDATA,
  output OUTPUT_TVALID,
  input OUTPUT_TREADY
);
```

5. Testbench

You are provided with a testbench to test your `Conv` system. This testbench will generate random data (X, W, B, Q, and S) and the expected output matrices. It will then provide the inputs to your module using the input AXI-Stream interface (with random timing on the `INPUT_TVALID` signal) and receive your module's outputs on the output AXI-Stream interface (with random timing on the `OUTPUT_TREADY` signal). The testbench will check your module's output values against the expected results and report any errors to you.

The testbench aims to check the correctness of the output of your design, but it does not directly check the efficiency or speed. (See Section 6 below for more discussion of this.)

You can find the testbench in the following two files:

```
/home/home4/pmilder/ese507/proj/part3/Conv_tb.sv
/home/home4/pmilder/ese507/proj/part3/test_helper.c
```

Copy these files into your `part3/` work directory. (Also copy `genParams3` and `simParams3` from the same location.) Before you use the testbench, please read the following brief explanation of how it works. Then read through the testbench files and read the comments.

The testbench module includes the following parameters, which will look familiar after your experience with the previous parts of the project:

- `WIDTH`: the number of bits for each input and output data word
- `R` and `C`: the number of rows and columns in the X matrix
- `K`: the number of rows/columns in the W matrix
- `TESTS`: the number of convolutions to simulate
- `INPUT_TVALID_PROB`: a decimal value that represents the probability that the testbench will assert `INPUT_TVALID` on any given cycle.
- `OUTPUT_TREADY_PROB`: a decimal value that represents the probability that the testbench will assert `OUTPUT_TREADY` on any given cycle.

The two `*_PROB` parameters should be between 0.001 and 1 inclusive, or if you select `0` the testbench will randomly pick a value at the beginning of the simulation (and tell you what it chose).

The testbench uses a SystemVerilog class called `testdata`. This class is similar to the class with the same name in the Part 2 testbench, but it also includes extra functions that will calculate the expected convolution output after generating random input data. If you would like to learn more about how this works, please see the code and comments in the testbench.

For each test, the testbench will randomize the test data and generate the expected corresponding output data. The testbench will feed the test input data into the system via the input AXI-Stream interface and receive the output data on the output AXI-Stream interface. The timing of the input's `TVALID` and the output's `TREADY` will be randomized based on the probability parameters described above. Similar to Part 2, the testbench will place random values on some signals when they are don't-cares.

In addition to the random tests, there are a small number of directed tests that force large values in the `X` and `W` matrices. This is intended to provide extra checking on the higher bits of the accumulator.

The testbench will check each output value and report any errors to the screen. It also will count the number of errors and stop running if the error count reaches 100. (You can easily modify this if you want to.) Additionally, the testbench will track the progress of the simulation; if progress stalls (for example, if a bug in your logic prevents the test from continuing) for one million cycles, the testbench will halt and print a message.

The testbench will also report the number of simulated cycles required to perform the given number of convolutions. We will discuss this more in Section 6 below.

Simulate your code using a variety of different parameters. Make sure your system works correctly across the range of `R`, `C`, `K`, and `WIDTH` values (defined in the Project Overview).

It's also important to adjust the `INPUT_TVALID` and `OUTPUT_TREADY` probabilities. For example, if `INPUT_TVALID` has high probability and `OUTPUT_TREADY` has low probability, your system will behave differently than if the probabilities were reversed. In the first case, the system will be slow at outputting data, so it will test what happens if your FIFO fills up and the system needs to stall. In the second case, you will be checking that your logic works correctly when it must frequently stall due to slow input loading. Make sure you simulate a variety of different scenarios. **In particular, make sure you run**

some simulations with very low `OUTPUT_TREADY` probability, since that is the only way to verify your logic works correctly when the Output FIFO fills up.

Run the testbench with the `simParams3` script, which works like the scripts in the prior parts. (Review the Part 1 description for more information, including what `[GUI]` means and how to reproduce a run from its random seed.)

```
./simParams3 [WIDTH] [R] [C] [K] [TVALID_PROB] [TREADY_PROB] [GUI]
```

where you will replace each bracketed item with the desired value of that parameter. Don't forget that if a `_PROB` value is set to 0, then the simulation will randomize its value.

As in the prior parts, `genParams3` only generates the `params.sv` file. You need it only if you are running the simulator by hand (for example, to reproduce a particular random seed):

```
./genParams3 [WIDTH] [R] [C] [K] [TVALID_PROB] [TREADY_PROB]
```

6. Throughput

We will characterize the speed of your designs based on throughput. Recall, throughput is a metric that quantifies the rate at which a system processes inputs or performs computations. We will measure throughput in terms of *convolutions per second*.

For example, if your system performs a convolution in 1000 cycles and we found through synthesis that it has a minimum clock period of 1ns, we would calculate its throughput as

$$\frac{1 \text{ conv}}{1000 \text{ cycles}} \times \frac{1 \text{ cycle}}{10^{-9} \text{ sec}} = 10^6 \text{ convs/sec}$$

where "conv" denotes one full convolution operation. So, this hypothetical system would compute 10^6 = 1 million convolutions per second.

The number of cycles required by your convolution system will depend on several factors:

- The values of parameters R, C, and K
- The value of the stride S for the specific input (which is limited by `MAXS`). Note that S affects the cycle count strongly, since it determines *how many* output values your system must compute.
- How frequently the testbench asserts `INPUT_TVALID` and `OUTPUT_TREADY`
- Whether the system needs to load a new W, B, S, and Q (when `new_W==1`) or use the previously stored ones (when `new_W==0`)

When we measure throughput, we will choose values for the hardware parameters `R`, `C`, `K`, and `WIDTH` (which dictate the characteristics of the hardware implementation) and values for the testbench parameters `INPUT_TVALID_PROB` and `OUTPUT_TREADY_PROB` (which control the testbench behavior we will simulate). We will measure the cycle count over many operations. `TESTS = 10000` is a good number for stable results. Although `S` and `new_W` are randomized for each test input, over the course of 10000 inputs their average values will be stable: the testbench randomly selects `S` between 1 and `MAXS` with equal probability of each value, and `new_W` is randomly 0 or 1 with equal probability.

How will you find the cycle count? Obviously, you aren't going to manually count millions of cycles. Instead, the testbench we provide will automatically count these cycles and show you the result at the end of the simulation. For example,

```
# Your system computed 10000 Convs in 73456788 cycles
```

So here, you would have 10,000 convolutions in 73,456,788 cycles. Next, you would synthesize this design to find the minimum clock period and compute the number of convolutions per second. Imagine that we synthesized this design and found the minimum clock period was 1.2ns. Then, we can compute the throughput averaged over 10000 test cases as:

$$\frac{10{,}000 \text{ convs}}{73{,}456{,}788 \text{ cycles}} \times \frac{1 \text{ cycle}}{1.2 \times 10^{-9} \text{ sec}} = 113{,}445 \text{ convs/sec}$$

In this case, our system's average throughput was 113,445 convolutions per second.

In the questions for the Final Report, you will be asked to do this calculation for some specific configurations.

7. Part 3 Submission

Simulate your design thoroughly with a variety of parameters to verify it works correctly. Don't forget to check extreme values of the `TREADY` and `TVALID` probabilities as discussed above.

Synthesize your design three times, with the following parameters:

- `WIDTH=12, R=9, C=8, K=5`
- `WIDTH=18, R=9, C=8, K=5`
- `WIDTH=24, R=16, C=17, K=9`

For each design, correct any synthesis problems you find, and find the maximum possible clock frequency.

Don't forget that when you synthesize, you need to set the parameter values in your top-level module. The `params.sv` file used in simulation only affects the testbench; to set the values in synthesis, you must make sure the default parameter values in your top-level module are set correctly. Save your synthesis reports with descriptive names and include them in your Part 3 submission.

Your Part 3 submission should include your code, your synthesis reports, your AI usage log, and a brief plaintext status report. Please see Section 6 in the Project Overview document, which explains the submission requirements and procedure.

8. Final Report

In your Final Report, include a clearly labeled section for Part 3, where you answer the following questions. More information about the Final Report can be found in Section 6 in the Project Overview document.

1. This part of the project required you to design a significant amount of control logic that interacts with the existing modules. Carefully and thoroughly document how your top-level system works, especially your control logic. Your documentation should allow the reader to fully understand how it works without looking at the code.

2. In Part 2, you wrote equations that described the number of cycles that the `input_mems` module requires (given `R`, `C`, and `K`) when `new_W==0` and `new_W==1`.

   Now, you should use your understanding of your system to write equations to describe the number of cycles for the entire convolution operation. (Again, you will have one equation for `new_W==0` and one for `new_W==1`.) Your equations should reflect the best-case number of cycles between when your system starts receiving inputs and when it is done with that computation and ready to receive a new set of inputs (where "best case" implies that `INPUT_TVALID` and `OUTPUT_TREADY` are always 1). Note that your equations here will depend on S as well as on R, C, and K. Formulate your answer based on your understanding of your design, and then run simulations to verify your equations.

   Based on your equations, is your system's performance limited by any one phase of execution? For example, if your input loading time is much higher than the compute time, this shows that the input loading time limits your speed much more than computation. On the other hand, if the system spends most of its time doing MAC operations on data, then the computational unit is the limiting factor. If the times are close to balanced, then this shows your system's performance is highly dependent on both of them. Importantly, keep in mind that this answer can change based on the values of R, C, K, and S. In other words, you may find you are limited by one factor when they are small, and another when they are large.

   Justify and explain your answers. Your answers to this question will be important in Part 4 of the project.

3. For the Part 3 submission, you synthesized the design with three sets of parameters:

   - `WIDTH=12, R=9,  C=8,  K=5`
   - `WIDTH=18, R=9,  C=8,  K=5`
   - `WIDTH=24, R=16, C=17, K=9`

   For each set of parameters, report the maximum clock frequency, minimum clock period, area, and power. For each, describe where the critical path is in the design. (Make sure you explain the critical path's logic fully; don't just list the start and end points.) If the different designs have meaningfully different critical path locations, explain or speculate as to why the location changes. You only need to report data for the smallest clock period you were able to find for each design.

4. Now, find the throughput of the second and third of the three designs you considered in question 3:

   - `WIDTH=18, R=9,  C=8,  K=5`
   - `WIDTH=24, R=16, C=17, K=9`

   (All of the following questions will refer to these two designs.)

   Find the throughput of each of the two designs under three different assumptions about testbench parameters `INPUT_TVALID_PROB` and `OUTPUT_TREADY_PROB`: 0.1, 0.5, and 1. (That is, do three simulations for each design, one where both `_PROB` parameters are 0.1, one where they are both 0.5, and one where they are both 1.)

   For each, record the number of clock cycles needed for 10,000 convolutions, as reported by the testbench with the parameters set appropriately. Then use those cycle counts along with the clock periods you found from synthesis to find the throughput in number of convolutions per second for each design under the three assumptions of the `_PROB` parameters.

In your report, make a table like Table 1 that shows the cycle counts and computed throughputs for all 6 scenarios (two designs with three sets of assumptions each). Make sure your tables include units (here and in all questions).

TVALID/ TREADY prob.	**Design 1 (`WIDTH=18`, `R=9`, `C=8`, `K=5`)**		**Design 2 (`WIDTH=24`, `R=16`, `C=17`, `K=9`)**	
	cycle count	**throughput (convs/sec)**	**cycle count**	**throughput (convs/sec)**
0.10				
0.50				
1.00				

Table 1: Cycle counts and computed throughputs for Design 1 and Design 2 across three TVALID/ TREADY probabilities.

5. The average delay of a system is the average amount of time that elapses between when the system starts a computation and when it finishes it. For the 6 scenarios evaluated in question 4, determine the delay in seconds (or ms, μs, ns, etc., as appropriate). Use the cycle counts you determined in the previous question and the clock period you determined in question 3. (Don't forget that the cycle counts reported in question 4 are for 10,000 convolutions; you need to find the average delay for a single convolution). Report these delays in a table.

6. The synthesis tool gives you an estimate of the power of your system. Use the power obtained from synthesis and the delays you computed in question 5 to determine the average energy your system consumes per convolution for each of the six scenarios you have evaluated in the previous questions. Report these values in a table.

   Remember: energy is measured in joules. Power = energy per time. 1 Watt = 1 Joule / 1 second.

7. A joint metric that combines the effects of area and speed in a single value is the *area-delay product*. The area-delay product is found by multiplying the area of the system times its delay. (Since these are both metrics that we want to minimize, lower area-delay products are better than higher ones.) Calculate the area-delay product of your system under these six scenarios and report the results in a table. Don't forget to include units in your answer.

8. If you worked with a partner, please carefully describe each partner's contribution to this part of the project. (If you did not work with a partner, skip this.)